MASTER PRODUCT REQUIREMENTS DOCUMENT

Campus Scheduling & Venue Management Platform

Institution
Chilika Institute of Technology (CIT)

Product scope
Academic scheduling, examinations, venue bookings, disruptions, AI assistance, campus mapping and indoor/outdoor navigation.

Document field	Value
Document ID	CIT-CAMPUS-PRD-001
Version	1.0 — comprehensive working draft
Status	Draft for team review; not institutionally approved
Prepared	31 October 2026
Primary build environment	Google AI Studio as implementation assistant; PRD remains source of truth

Important: This document consolidates the discussions so far. It distinguishes confirmed requirements, proposed design and open institutional-policy questions. OPEN and PROPOSED items must not be treated as approved policy.

Working principle: define product rules and data contracts first; implement in small verified increments; never let AI-generated code silently redefine the system.

Document Control & Reading Guide

This is the central product and engineering specification for the CIT platform. It is designed for the project team and for use as source material in Google AI Studio.

Label	Meaning	Instruction
CONFIRMED	Explicit in the problem brief or clearly decided in discussion.	Preserve unless formally changed.
PROPOSED	Recommended product or engineering design.	Use as working baseline; review before final approval.
OPEN	Needs confirmation from CIT, dataset, team or judges.	Do not invent; keep configurable or ask.
OUT OF SCOPE	Excluded from current release.	Do not implement without scope change.

How to use this PRD

- Give AI Studio the Master PRD and only the relevant technical specification for each implementation task.
- Require repository inspection before code edits; preserve working code and explain proposed changes.
- Implement one feature at a time with schema/API impact, tests and acceptance evidence.
- The approved PRD and contracts outrank assumptions generated by the coding assistant.
- Do not ask AI Studio to generate the entire application in one prompt.

Known limitations

Exact starter-pack schemas, validator implementation, CIT approval policy, campus map assets and team programming proficiency must be checked against actual source files or confirmed by the institution.

1. Executive Summary

CIT requires a unified platform to coordinate academic timetables, examinations, institutional and club venue requests, operational disruptions and campus wayfinding. It must support distinct experiences for administrators, coordinators, faculty and students while maintaining a single validated institutional source of truth.

This is not merely a calendar or booking form. It is a rules-driven institutional operations system. Scheduling and booking decisions must be deterministic, explainable and backend-validated. AI may interpret natural language, retrieve context and prepare proposals, but it must not independently approve or publish institutional changes.

1.1 Product outcomes

- Generate a weekly timetable satisfying hard constraints and the supplied validator.
- Schedule the following week's mid-semester exams with exam-slot and room-capacity rules.
- Convert plain-English/Hinglish venue requests into editable structured requests.
- Process venue approvals using role-category eligibility, priority ordering, conflict checks and explanations.
- Handle faculty leave, room closure and new sessions by proposing valid changes while minimising unnecessary movement.
- Provide group, faculty and room views, pending approvals, conflicts and room utilisation.
- Provide an AI assistant whose proposals are always checked by deterministic backend validation.
- Provide a simple 2D campus navigator for outdoor and indoor places, integrated with timetable room records.
- Allow administrators to import the starter dataset and export required JSON.

1.2 Product principles

- Single source of truth; backend authority; human approval; explainable decisions; complete audit history.
- Immutable versions for published schedules and campus maps.
- Simple student experience; powerful but controlled admin tools.
- Modular monolith first; avoid unnecessary distributed-system complexity.

Product promise: Explain what the system did, why it did it, which rules it checked, who authorised the change and what changed.

1.3 Recommended architecture baseline

PROPOSED: React + TypeScript frontend; Python + FastAPI backend; PostgreSQL; SQLAlchemy + Alembic; Google OR-Tools CP-SAT; Gemini for constrained AI assistance; MapLibre for outdoor maps and SVG/graph routing for indoor maps. A Python worker handles long-running solver and map-processing jobs.

Payload CMS was considered. It is a capable Next.js/TypeScript backend with an admin panel, authentication, access control, APIs and PostgreSQL support. The central optimisation and validation logic is Python-oriented, however. Using Payload as the primary backend would likely require a separate Python solver service. FastAPI is therefore the simpler baseline unless team skill strongly favours TypeScript and the team accepts that split.

2. Product Scope & Boundaries

2.1 Modules in scope

ID	Module	Scope
MOD-001	Authentication & access	Identity, roles, scopes and permissions.
MOD-002	Institutional data	Import, validate, version rooms, faculty, groups, courses and source records.
MOD-003	Timetable	Generate, validate, inspect, manually adjust and publish weekly schedules.
MOD-004	Examinations	Schedule exams, allocate rooms and inspect conflicts.
MOD-005	Venue management	Natural-language requests, eligibility, priority, review, alternatives and booking.
MOD-006	Disruptions	Record unavailability/closures/new sessions, propose reschedules and approve application.
MOD-007	AI assistant	Interpret questions/changes, retrieve context, propose actions and explain validated outcomes.
MOD-008	Campus Navigator	Search campus places and generate outdoor/indoor routes.
MOD-009	AI Campus Map Builder	Ingest assets, propose geometry/semantics and prepare admin review.
MOD-010	Analytics & reports	Conflicts, approvals, room utilisation, summaries and exports.
MOD-011	Notifications	In-app notifications for submissions, decisions and changes.
MOD-012	Audit & administration	Audit history, configuration, dataset versions and operational controls.

2.2 Out of scope for current MVP

- BIM-grade building modelling or a full architectural authoring tool.
- AR navigation, autonomous indoor positioning, BLE/Wi-Fi triangulation or guaranteed indoor GPS.
- Autonomous AI approval, automatic map publication or unrestricted AI database writes.
- A full GIS editor; unrestricted public map editing; unverified map publication.
- Microservices and Kubernetes without a demonstrated need.

2.3 Release levels

Release	Target
Hackathon P0	Dataset import; timetable and validator; bookings; disruption proposal; core dashboards; constrained AI; core campus map demonstration; required exports.
MVP P1	Robust permissions, versioning, notifications, map upload/review/publish, analytics and hardening.
Extended P2	Saved preferences, richer reports, map editing assistance, QR start points and more institutional workflows.
Production P3	SSO, monitoring, disaster recovery, policy integrations, accessibility audit and load testing.

2.4 Scope change control

Every new feature or changed rule needs an identifier, reason, affected modules, data/API impact, risk, acceptance criteria and explicit project-owner approval. AI Studio must not silently broaden scope.

3. Users, Roles & Authority

Six user categories have been discussed. A user account may hold multiple roles; roles are not mutually exclusive. Permissions are scoped by campus, department, club, resource and action. Actual CIT role assignments remain OPEN.

Role	Responsibilities	Proposed permissions
Super Admin	Institution-wide platform governance.	Global settings, roles, imports, audit access and policy configuration.
Campus Admin	Campus operations and scheduling oversight.	Campus schedules, venue decisions, disruptions, map review/publication where authorised.
Department Coordinator	Department coordination.	Department schedules, faculty/group coordination and delegated requests.
Club Coordinator	Club events and venue coordination.	Create/track club requests, respond to alternatives and manage club event information.
Faculty	Teaching and faculty tasks.	Own timetable, submit unavailability/leave or venue requests where policy permits, assistant and navigation.
Student	Student-facing schedule and navigation.	Own group timetable, exams/notices, campus search and navigation.

3.1 Role principles

- Every protected action is checked against authenticated identity and active permission on the server.
- Frontend hiding is for usability only; it is not authorisation.
- Multiple-role users act under a clear effective role/context; logs record that context.
- Least privilege applies. Self-approval is prohibited for bookings and workflows requiring separation of duties.
- No role can override hard scheduling constraints or publish unvalidated data.
- An Examination Authority permission may be assigned to an authorised admin; do not invent a separate role until CIT confirms.

3.2 Proposed authority matrix

Action	Super Admin	Campus Admin	Dept Coord.	Club Coord.	Faculty	Student
View published schedule	Yes	Yes	Scoped	Scoped	Relevant	Own group
Generate/publish timetable	Yes	Yes*	No	No	No	No
Submit venue request	Yes	Yes	Yes	Yes	Policy	No
Review venue request	Policy	Yes*	Scoped*	No self-review	No self-review	No
Submit disruption	Yes	Yes	Scoped*	No	Policy	No
Approve reschedule	Policy	Yes*	Scoped*	No	No	No
Upload/review/publish map	Yes	Yes*	No	No	No	No
Manage roles/audit	Yes	Scoped*	No	No	Own actions*	No

* PROPOSED; exact authority and delegation are OPEN. Scoped means only within assigned campus, department, club or resource.

3.3 Approval governance

- Workflows may be single-level or multi-level depending on request type and institutional policy.
- Example only: Club Coordinator submits → reviewer checks completeness → authorised Campus Admin decides. Not confirmed CIT policy.
- Every decision records reviewer, effective role, timestamp, reason and evidence.
- Rejection requires a reason; an alternative offer is not an approved booking.
- Delegation, escalation, expiry and absence coverage are OPEN policy settings.

4. Functional Requirements

Priority: P0 = hackathon/judging essential; P1 = important MVP; P2 = extension; P3 = later production. Each feature needs owner, API/data contract, permission, failure behaviour and acceptance tests.

4.1 Authentication & access (MOD-001)

- **FR-AUTH-001 P0:** Authenticate users and establish server-verified identity.
- **FR-AUTH-002 P0:** Resolve roles and scopes from trusted database records.
- **FR-AUTH-003 P0:** Enforce permissions on every protected endpoint and state-changing operation.
- **FR-AUTH-004 P1:** Support multiple roles and explicit active context.
- **FR-AUTH-005 P1:** Audit denied sensitive actions and role changes.

4.2 Institutional data/import (MOD-002)

- **FR-DATA-001 P0:** Import starter pack without hardcoding sample values.
- **FR-DATA-002 P0:** Validate files, identifiers, references, enums and duplicate source IDs before activation.
- **FR-DATA-003 P0:** Preserve source IDs such as F07, CR-102 and session IDs.
- **FR-DATA-004 P0:** Store immutable dataset version, timestamp, actor, checksum and validation report.
- **FR-DATA-005 P0:** Read canonical JSON configuration, including calendar, priority_order and role_category_cap; supporting CSVs may omit root config.
- **FR-DATA-006 P1:** Support safe re-import with diff/impact preview.

4.3 Timetable (MOD-003)

- **FR-SCH-001 P0:** Generate weekly timetable for all required teaching sessions.
- **FR-SCH-002 P0:** Enforce all hard constraints before valid/publish state.
- **FR-SCH-003 P0:** Provide group, faculty and room views.
- **FR-SCH-004 P0:** Support authorised manual assignment with same hard constraints.
- **FR-SCH-005 P0:** Explain hard-rule failures and soft-score contributions.
- **FR-SCH-006 P1:** Save versions and compare draft/published schedules.
- **FR-SCH-007 P1:** Publish only validated version; preserve rollback history.

4.4 Examinations (MOD-004)

- **FR-EXAM-001 P0:** Schedule mid-semester exams for the next exam week.
- **FR-EXAM-002 P0:** Enforce permitted exam slots and half room-capacity rule.
- **FR-EXAM-003 P0:** Allow one exam to use multiple rooms where necessary.
- **FR-EXAM-004 P0:** Prevent student-group and room overlap and respect exam availability.
- **FR-EXAM-005 P1:** Support review and authorised exam publication.

4.5 Venue requests & bookings (MOD-005)

- **FR-BOOK-001 P0:** Parse plain-English/Hinglish requests into structured drafts.
- **FR-BOOK-002 P0:** Let requester inspect/edit parsed details before submission.
- **FR-BOOK-003 P0:** Enforce role-category caps and priority order.
- **FR-BOOK-004 P0:** Exclude cancelled requests from approval consideration.
- **FR-BOOK-005 P0:** Enforce self-approval rule and room/time/capacity/group/schedule conflicts.
- **FR-BOOK-006 P0:** Support approve, reject and offer-alternative with explanation.
- **FR-BOOK-007 P0:** Same-category tie breaks by earlier submitted_at.
- **FR-BOOK-008 P1:** Notify requester after committed decision and provide history.

4.6 Disruptions/rescheduling (MOD-006)

- **FR-DISR-001 P0:** Record faculty unavailability, room closure and new-session requirements.
- **FR-DISR-002 P0:** Propose valid reschedule while moving as few existing sessions as possible.
- **FR-DISR-003 P0:** Show all proposed additions/removals/changed assignments before approval.
- **FR-DISR-004 P0:** Explain infeasibility.
- **FR-DISR-005 P0:** Apply only after authorised human approval.
- **FR-DISR-006 P0:** Applied disruption is permanent and recurring weekly as specified in the CIT brief.
- **FR-DISR-007 P1:** Preserve baseline timetable and proposal base version.

4.7 AI assistant (MOD-007)

- **FR-AI-001 P0:** Answer role-authorised schedule questions from current institutional data.
- **FR-AI-002 P0:** Convert natural language to structured drafts, not direct writes.
- **FR-AI-003 P0:** Use only explicitly defined backend tools.
- **FR-AI-004 P0:** Validate every proposed action with deterministic logic.
- **FR-AI-005 P0:** Require user confirmation and institutional approval where applicable.
- **FR-AI-006 P1:** Explain passed, failed and uncertain constraints.

4.8 Navigator/map builder (MOD-008/009)

- **FR-NAV-001 P0:** Search buildings, rooms, facilities and places.
- **FR-NAV-002 P0:** Route using verified outdoor paths and indoor graph edges.
- **FR-NAV-003 P0:** Integrate “Navigate to class” with next class from published timetable.
- **FR-NAV-004 P0:** Reuse existing room IDs; do not duplicate room records.
- **FR-MAP-001 P1:** Admin upload of PDFs, scanned images, satellite/map images and existing map data.
- **FR-MAP-002 P1:** Process assets into proposed structured map data.
- **FR-MAP-003 P1:** Admin reviews geometry, room labels, entrances and walkability.
- **FR-MAP-004 P1:** Publish immutable map versions and support rollback.

4.9 Operations (MOD-010/011/012)

- **FR-OPS-001 P0:** Show pending approvals, conflicts, timetable status and room utilisation.
- **FR-OPS-002 P0:** Export exact required JSON structures and paths.
- **FR-OPS-003 P0:** Audit sensitive changes and decisions.
- **FR-OPS-004 P1:** Notify affected users after successful commit.

5. CIT Scheduling & Booking Rules

Source note: Rules below reflect the CIT problem/data discussion. The official validator and actual starter files remain authoritative. Inspect them before implementation.

5.1 Teaching calendar

Period	Start	Use
P1	08:00	Teaching
P2	09:00	Teaching; permitted 3-hour lab start
P3	10:00	Teaching
P4	11:00	Teaching
P5	12:00	Teaching
Lunch	13:00–14:00	No teaching
P6	14:00	Teaching; permitted 3-hour lab start
P7	15:00	Teaching
P8	16:00	Teaching
P9	17:00	Teaching
P10–P13	18:00–22:00	Event-only

Teaching days: Monday–Friday; nine teaching periods per day. Events may use P10–P13. Labs last three hours and may start only at P2 or P6. Teaching cannot cross lunch.

5.2 Exam calendar

Exams may be Monday–Saturday. AM 09:30–11:30; PM 14:00–16:00. Exam room capacity is half normal capacity. One exam may be allocated across multiple rooms. Verify additional rules in official data.

5.3 Hard constraints

ID	Rule	Implication
HC-01	Every required session exactly once.	No missing/duplicate assignments.
HC-02	Faculty cannot overlap.	No overlapping faculty assignments.
HC-03	Mandatory student groups cannot overlap.	No group double-booking.
HC-04	Rooms cannot overlap.	No room double-booking.
HC-05	Capacity sufficient.	Attendance <= usable room capacity.
HC-06	Room type/features match.	Required type/features enforced.
HC-07	Respect faculty/room unavailability.	Exclude unavailable intervals.
HC-08	Accessibility requirements met.	Accessible resource/path where required.
HC-09	No teaching crosses lunch.	Intervals cannot intersect lunch.
HC-10	Labs are 3h, start P2 or P6.	Duration and start domain enforced.
HC-11	Elective basket alignment.	Same basket_slot, same day/period, different rooms, no mutual group conflict.
HC-12	Faculty daily limit 4h in CIT data.	Count relevant teaching periods per faculty/day.
HC-13	Approved booking cannot conflict with classes/other bookings.	Recheck room and mandatory group.
HC-14	Fixed slots preserved.	Keep fixed assignments unless authorised release.

5.4 Soft constraints / recorded weights

Weights below were recorded in discussion and must be checked against validator.py before implementation.

ID	Soft objective	Recorded weight
SC-01	Faculty avoid-period preference	2 per avoid period
SC-02	Student-group free-period gap	1 per gap
SC-03	Group exceeds 6 teaching hours/day	3 per excess group-hour
SC-04	Additional same-course lectures/day	3 per additional lecture
SC-05	Room use below 40%	1 per underutilised session

ID	Soft objective	Recorded weight
SC-06	P9 usage	1 per P9 session

Hard constraints are feasibility rules, not weighted preferences. Soft objectives are optimised only after hard feasibility is maintained. Explain score components to admins.

5.5 Venue priority and role-category caps

Priority	Category	Role cap	CAT_WEIGHT
1	Exams	Not general booking category; highest operational priority	—
2	Regular classes	Must be preserved	—
3	Institute event	Admin → institute_event	5
4	Guest lecture	Faculty → guest_lecture	4
5	Department event	Department Coordinator → department_event	3
6	Club event	Club Coordinator → club_event	2

Same-category tie: earlier submitted_at wins. Cancelled requests are never approved. Load priority order and role caps from canonical JSON/config where present; do not hardcode.

5.6 Starter dataset facts

Item	Recorded summary
Dataset ID	cit-public-v1
Rooms	27
Faculty	40
Student groups	14
Courses	62
Sessions	189
Weekly teaching load	243 teaching hours
Venue requests	25 (one cancelled)
Midsem exams	41
Feasibility note	README reportedly states a zero-hard-violation timetable exists.

JSON is canonical; CSV files are supporting inputs and may omit calendar, priority_order and role_category_cap. Preserve source IDs. Judge may reset the app and provide different change files.

5.7 Validator and exports

- validator.py is authoritative for scoring and hard-rule semantics.
- Run the official validator against generated output, not only internal checks.
- Application must produce exports/schedule.json and exports/schedule_after.json.
- Exports must be generated from validated persisted versions; never hand-edit.
- A new judge dataset must not require code changes.

6. Approval, Authority & Workflow Management

Approval is cross-cutting, not merely a screen. Every state-changing feature must specify who can submit, review, approve, reject, request changes, publish and override. Exact chains are policy-configurable and require CIT confirmation.

6.1 Workflow types

Type	Examples	Control
Administrative	Venue booking, exam/timetable/map publication.	Authorised human decision with rationale.
Operational	Leave, room closure, new session, reschedule.	Proposal validated, reviewed and approved before applying.
AI-assisted	Booking parse, assistant proposal, map extraction.	AI drafts; backend validates; human decides.

6.2 Common lifecycle

```
DRAFT -> SUBMITTED -> UNDER_REVIEW
  -> APPROVED
  -> REJECTED
  -> CHANGES_REQUESTED -> RESUBMITTED
  -> ALTERNATIVE_OFFERED -> ACCEPTED / DECLINED
  -> CANCELLED (where policy permits)
```

This is a common pattern, not a requirement that every module use every state. Each module must define valid transitions. An alternative offer is not an approval.

6.3 Approval record

- Request ID/type; requester; effective role/scope; submitted timestamp; status.
- Assigned reviewer/stage; decision; decision reason; decision maker and timestamp.
- Evidence/attachments; conditions/alternative; base version; complete transition history.

6.4 Safeguards

- Recheck authority, scope, self-approval and current resource availability at decision time.
- Hard constraints cannot be overridden by approval.
- Rejection requires reason; material edits trigger revalidation/reapproval.
- Stale proposals must be checked against current dataset/timetable/map version.
- Decision, reservation/state mutation and audit event commit atomically.
- Notify only after transaction commit; use idempotency for retries.
- Every state transition is auditable.

6.5 Booking approval transaction

```
BEGIN
  authenticate actor; resolve active role and scope
  lock relevant resource rows consistently
  load request and current dataset/timetable version
  verify not cancelled/already decided
  verify reviewer authority and no self-approval
  recheck priority, capacity, room type, time/group/faculty conflicts
  if valid: write decision + allocation + audit event
  else: return structured conflict details
COMMIT
send notification after commit
```

Use a transaction with consistent row locking for contested resources; exclusion constraints can provide defence in depth where interval representation supports them. Do not rely on frontend availability checks.

6.6 Publication controls

- Solver proposals record base dataset and timetable version.
- Before publication, verify base version is still current or rebase/revalidate.
- Published timetable/map versions are immutable; changes create a new version.
- Map publication requires review of room labels, entrances, floor connections and walkability.

6.7 OPEN policy questions

- Which request types require single-level versus multi-level approval?
- Can Department Coordinators approve or only recommend/review venue requests?
- Who has final authority to publish timetable, exam and map versions?
- What are deadlines, escalation, expiry, delegation and absence-coverage rules?
- Which leave/unavailability requests require approval versus informational recording?

7. System Architecture

7.1 Recommended stack

Layer	Technology	Responsibility
Frontend	React + TypeScript + Vite	Role-based web UI.
UI	Tailwind CSS + shadcn/ui	Consistent accessible components.
Client data	TanStack Query; Zustand only where needed	Server-state and limited local state.
API	Python + FastAPI + Pydantic	Business rules, validation and contracts.
Database	PostgreSQL	Relational data and transactions.
ORM/migrations	SQLAlchemy 2.x + Alembic	Data access and schema evolution.
Optimisation	Google OR-Tools CP-SAT	Timetable and rescheduling.
AI	Gemini structured output/function calling	Language understanding and tool orchestration.
Outdoor map	MapLibre + GeoJSON	Map rendering and outdoor geometry.
Indoor map	SVG/React + graph routing	Simple floor plans and indoor routes.
Jobs	Python worker; Redis + RQ/Celery if needed	Long-running jobs.
Tests	Pytest + frontend tests + official validator	Regression and acceptance.

7.2 Payload decision

Payload is a strong TypeScript/Next.js backend framework with generated admin UI, authentication, access control, APIs and PostgreSQL support. It fits content-heavy systems and admin-managed records. CIT's difficult domain logic—constraint programming, rescheduling and parts of map processing—is Python-oriented. Making Payload primary likely introduces a separate Python solver service and cross-service contracts.

Recommendation: FastAPI as the main application API; build the admin interface in React. Reconsider Payload if the team is significantly stronger in TypeScript/Next.js and accepts a Python solver service. Team skill is still OPEN and should be confirmed before locking the architecture.

7.3 Modular monolith

```
React / TypeScript
       |
FastAPI application
  |-- auth & permissions
  |-- institutional data / imports
  |-- timetable / exams
  |-- bookings / approvals
  |-- disruptions / rescheduling
  |-- AI orchestration
  |-- campus navigation / map assets
  |-- analytics / exports / notifications
       |
       +-- PostgreSQL
       +-- Python worker (solver / map processing)
       +-- Gemini API (tool-limited)
       +-- Object storage (map assets)
```

Keep modules in one deployable backend with clear internal service boundaries. Use a separate worker process for long jobs; avoid microservices without demonstrated need.

7.4 Data flow

```
User -> React -> FastAPI
     -> authenticate + scope check
     -> Pydantic validation
     -> domain service / deterministic rules
     -> PostgreSQL transaction + audit
     -> response; notify after commit

AI-assisted:
User text -> Gemini structured draft/tool call
 -> backend retrieves authorised context
 -> deterministic validation
 -> user review/edit
 -> approval workflow
 -> transaction applies action
```

7.5 Deployment proposal

- Frontend: Vercel or equivalent.

- FastAPI API and worker: Python-capable application host.
- Database: managed PostgreSQL; PostGIS when spatial queries are needed.
- Map source files: controlled object storage.
- Secrets in environment/secret manager, never committed.
- Provider, region, budget and free-tier constraints remain OPEN.

8. Database Design Blueprint

PostgreSQL is recommended because the data is relational, transactions and constraints matter, and schedule/booking conflicts must be safe under concurrent writes. This is a conceptual schema; final physical schema requires review and Alembic migrations.

8.1 Principles

- Internal UUID primary keys; preserve external/source IDs in unique columns.
- Use foreign keys for institutional relationships; do not rely only on unchecked JSON.
- Use explicit campus-local timezone policy; do not bury timezone assumptions in code.
- Separate recurring weekly slots from dated exceptions where practical.
- Version datasets, timetable publications, exam publications and maps.
- Use documented states, created/updated metadata and audit references.
- Soft-delete only when recovery/history is needed; retain immutable audit history.

8.2 Identity and institutional tables

Table	Key fields / purpose
users	id, email, display_name, auth_subject, status.
roles / permissions	Role and permission codes, resource/action definitions.
user_roles	user_id, role_id, scope_type, scope_id, validity interval.
role_permissions	role_id, permission_id, scope constraints.
departments	id, source_id, name, code.
faculty_profiles	user_id, faculty_source_id, department_id.
student_profiles	user_id, student_source_id, group_id.
student_groups	id, source_id, name, department_id, size.
student_group_members	student_id, group_id, effective dates if required.
clubs	id, source_id, name, scope references.

8.3 Academic and timetable tables

Table	Key fields / purpose
courses	id, source_id, code, title, department_id.
rooms	id, source_id, code, building_id, capacity, room_type, features, accessibility.
sessions	id, source_id, course_id, type, duration, required_size, fixed flag.
session_groups	session_id, group_id, requirement_type.
session_requirements	session_id, room type/features/equipment/accessibility.
calendar_periods	day_of_week, period_code, start/end, period_type.
faculty_assignments	session_id, faculty_id, assignment type.
faculty_unavailability	faculty_id, interval/recurrence, reason, approval state.
room_unavailability	room_id, interval/recurrence, reason, state.
faculty_time_preferences	faculty_id, day/period, preference type, weight.
timetable_versions	id, dataset_version_id, version, status, solver metadata, published_at.
session_assignments	version_id, session_id, room_id, faculty_id, day, start, duration.

8.4 Exams, bookings and approvals

Table	Key fields / purpose
exam_definitions	source_id, course, exam type, duration, eligible groups.
exam_group_requirements	exam_id, group_id, candidate_count.
exam_assignments	exam version, exam, room, date, slot, allocated capacity.
booking_requests	requester, category, interval, attendance, purpose, status, submitted_at.
booking_decisions	request, decision, reason, actor and timestamp.
booking_allocations	request, room, interval, allocation status.
booking_priority_rules	category, priority rank, weight and role cap.
approval_records	entity, stage, assigned reviewer, state, decision metadata.
approval_transitions	approval, from/to state, actor, reason, timestamp.

8.5 Disruptions and operations

Table	Key fields / purpose
disruptions	type, affected resource, interval/recurrence, status, submitter.
reschedule_proposals	disruption, base timetable version, solver status, score, status.
reschedule_assignment_changes	proposal, session, old assignment, proposed assignment.
solver_jobs	job type, input version, status, progress, timestamps, error.
dataset_versions	dataset code, checksum, source metadata, importer, status.
audit_logs	actor, action, entity, before/after refs, timestamp, request ID.
notifications	recipient, type, payload, read_at, created_at.
export_jobs	type, source version, status, file reference, created_at.

8.6 Campus map tables

Table	Key fields / purpose
campuses	name, geographic reference, timezone.
buildings	campus, code, name, outdoor footprint.
floors	building, floor code, display name, level index.
map_assets	uploader, filename, MIME type, storage key, checksum, status.
map_versions	campus, version, status, source asset IDs, publication time.
spaces	building/floor, linked_room_id nullable, type, label, geometry.
doors / entrances	floor/building, point/geometry, access and verification state.
navigation_nodes	map version, floor, coordinate frame, x/y or lat/lon, type.
navigation_edges	from/to nodes, distance, edge type, accessibility, directionality.
map_processing_jobs	asset, processor version, status, output ref, error.
map_review_items	version, item type, geometry, confidence, review state, reviewer.
map_publications	version, publisher, timestamp, rollback reference.

8.7 Database constraints

- Unique source IDs within source/entity namespace; foreign keys for academic relationships.
- Positive durations/capacities and constrained status values.
- Prevent overlapping active reservations for the same room where interval types permit exclusion constraints.
- Use transactions and consistent row locks for approval-time conflict checks.
- Index timetable version/day/period, room/time, faculty/time, group/time, request status/submitted_at and spatial columns.
- Application validator remains necessary; no single DB constraint enforces every multi-resource scheduling rule.

8.8 Import workflow

```
Upload -> validate manifest/schema -> parse canonical JSON + CSV
 -> validate IDs/references/duplicates -> stage dataset version
 -> produce import report -> domain consistency checks
 -> authorised activation -> retain previous version/checksum
```

9. API & Integration Specification

The following is a starting route map, not a claim that routes already exist. Final request/response schemas must be defined in FastAPI OpenAPI and kept in sync with frontend types.

9.1 Conventions

- Use versioned routes such as /api/v1/... .
- Use consistent JSON errors with code, message, field details and request/trace ID.
- Paginate lists and filter by status, date, scope and resource.
- Use idempotency keys for retryable state-changing operations.
- Use optimistic version checks for edits to drafts/proposals.
- Never expose SQL errors, secrets or stack traces to clients.

9.2 Endpoint groups

Area	Example endpoints
Auth	POST /auth/session; GET /me; GET /me/permissions
Dataset	POST /admin/datasets/import; GET /admin/datasets; GET /admin/datasets/{id}/validation
Timetable	POST /admin/timetables/generate; GET /timetables/{id}; GET /timetables/views/group/{id}; POST /admin/timetables/{id}/publish
Exams	POST /admin/exams/schedule; GET /exams; POST /admin/exams/{id}/publish
Bookings	POST /bookings/parse; POST /bookings; GET /bookings; POST /bookings/{id}/decision; POST /bookings/{id}/alternative
Disruptions	POST /disruptions; POST /disruptions/{id}/proposals; GET /proposals/{id}; POST /proposals/{id}/approve; POST /proposals/{id}/apply
AI	POST /assistant/message; POST /assistant/proposals/{id}/validate; POST /assistant/proposals/{id}/confirm
Campus	GET /campus/places/search; GET /campus/places/{id}; POST /campus/navigation/route; GET /students/me/next-class
Map admin	POST /admin/campus-map/uploads; /process; /review-items; /validate; /publish; /rollback
Operations	GET /admin/approvals; /conflicts; /analytics/room-utilisation; POST /admin/exports

9.3 Example booking contract

```
POST /api/v1/bookings
Idempotency-Key: <client-generated-key>
{
  "category": "club_event",
  "purpose": "Robotics club workshop",
  "requested_start": "2026-11-10T18:00:00+05:30",
  "requested_end": "2026-11-10T20:00:00+05:30",
  "expected_attendance": 65,
  "preferred_room_types": ["classroom", "seminar_hall"],
  "required_features": ["projector"]
}
```

9.4 Long-running jobs

Timetable generation and map processing should return a job ID. Job states: QUEUED → RUNNING → VALIDATING → COMPLETED/FAILED/CANCELLED. Solver result is separate: OPTIMAL, FEASIBLE, INFEASIBLE, UNKNOWN or MODEL_INVALID.

9.5 Contract gates

- Every endpoint declares authentication, permission, transaction and audit requirements.
- Every response defines stable fields and nullable/optional semantics.
- Every error path is documented and tested.

10. Scheduling & Optimisation Engine

10.1 Solver design

Use Google OR-Tools CP-SAT as the proposed optimisation engine. Model session assignment variables, room/faculty/group conflicts, durations, availability, fixed slots, room capabilities, basket alignment and objective penalties. Enforce hard constraints in the model and independently check the candidate after solving.

10.2 Solve pipeline

```
Load active dataset version
 -> normalise sessions, resources, calendar and rules
 -> build CP-SAT model
 -> add hard constraints and soft objectives
 -> solve with configured time limit
 -> inspect status
 -> materialise candidate assignments
 -> independent application validator
 -> official CIT validator
 -> save draft version and score report
 -> authorised review -> publish
```

10.3 Solver status handling

Status	Meaning	Behaviour
OPTIMAL	Feasible solution proven optimal.	Show solution and optimality.
FEASIBLE	Valid solution found; optimality unproven.	Show solution with caveat.
INFEASIBLE	No feasible solution under encoded constraints.	Show diagnostics; do not publish.
UNKNOWN	No conclusive result before stop/limit.	Do not claim infeasible.
MODEL_INVALID	Model failed validation.	Treat as engineering defect.

10.4 Rescheduling objective

Preserve unaffected assignments and minimise the number/weighted cost of moved sessions while satisfying all hard constraints. Keep fixed sessions unless an authorised disruption explicitly releases them. Show old versus proposed assignments and the reason for every move.

10.5 Infeasibility explanation

- Identify affected faculty, group, room, time and rule categories.
- Distinguish proven INFEASIBLE from time-limited UNKNOWN.
- Offer only policy-safe relaxations, such as adding a room or changing a time window.
- Never silently weaken hard constraints to produce a timetable.

10.6 Manual scheduling

Manual drag/drop or form assignment must call the same domain validator as the solver. Backend is authoritative; failed assignment returns structured conflict codes and affected resources.

11. AI Integration & Guardrails

11.1 AI role

Gemini is an interpretation and assistance layer, not the source of truth. It may parse language, retrieve authorised context, draft requests, propose changes and explain validated outcomes. It may not write arbitrary database records, bypass permissions, approve requests or publish versions.

11.2 Tool design

Tool	Purpose	Restriction
get_campus_context	Read campus configuration.	Read-only; scoped.
list_buildings / floors	Read map structure.	Authorised versions only.
get_room_records	Read canonical room records.	Read-only; preserve IDs.
get_schedule	Read schedule by group/faculty/room.	Scope enforced server-side.
parse_booking_request	Convert text to draft.	No submission/approval.
validate_booking	Check category, time, capacity, conflicts.	Backend deterministic.
propose_reschedule	Create solver job.	No automatic apply.
validate_map_asset	Process map candidate.	Draft only.
propose_geometry_change	Create map suggestion.	Never publish.
validate_map_version	Check topology/semantics.	No approval.
prepare_publish_check	Summarise blockers/approvals.	No publication side effect.

11.3 AI action lifecycle

```
User text -> intent + structured extraction
 -> clarify missing mandatory fields
 -> create draft proposal
 -> user reviews/edits
 -> backend validates identity, permissions and rules
 -> approval workflow where required
 -> authorised human decision
 -> transaction commits -> audit + notify
```

11.4 Guardrails

- Tool schemas are explicit, typed and narrow; every tool independently checks identity and scope.
- No unrestricted SQL, shell, filesystem or arbitrary network tools.
- Treat uploaded documents/maps/user text as untrusted input.
- Validate AI output with Pydantic and domain validators.
- If data is missing or uncertain, ask a clarification; do not fabricate.
- Do not claim room capacity, access, policy or route accessibility without verified data.
- Log model/tool/proposal/validation metadata without unnecessarily retaining sensitive prompts.
- Human confirmation is required for consequential actions.

11.5 Natural-language booking example

“We need the seminar hall next Wednesday from 6 to 8 for 60 people.” Extract date, time, purpose, attendance and room preference; show editable draft; ask for missing fields; submit only after user confirms. Backend derives allowed category from trusted role configuration, not the category claimed in text.

12. Campus Navigator & AI Map Builder

12.1 Product intent

A simple 2D wayfinding feature for outdoor and indoor campus navigation. A student searches a room, building or facility, sees its location and receives a route. “Navigate to class” resolves the next scheduled classroom from the same timetable room ID.

12.2 Coordinate systems

- Outdoor geometry uses WGS84 longitude/latitude and may be rendered with MapLibre/GeoJSON.
- Indoor floor plans use a local per-floor x/y frame, such as a normalised source plan or SVG viewBox.
- Do not treat indoor image pixels as WGS84 GeoJSON coordinates.
- A verified building entrance connects outdoor graph to indoor entrance node.
- Floor transitions use explicit stair/lift nodes and accessibility attributes.

12.3 Navigation graph

Nodes represent intersections, doors, entrances, room access points, stairs and lifts. Edges represent verified walkable connections with distance, type, directionality and accessibility. Use Dijkstra or A*; NetworkX is adequate for MVP, while PostGIS supports spatial lookup and future scale.

12.4 Map source ingestion

Admin may upload PDF, scanned floor plan, satellite/map image or existing vector/map data. Retain original file; classify source; extract candidate geometry/labels; generate draft map. AI/CV may suggest walls, doors, room boundaries, labels and entrances, but an admin must verify walkability and connections.

12.5 Processing pipeline

```
UPLOAD
 -> file/type/size checks + checksum
 -> classify source
 -> preprocess (deskew/crop/enhance/rasterise)
 -> extract geometry and text (OCR/CV/parser)
 -> semantic interpretation (room/corridor/door/stairs/lift)
 -> normalise canonical map schema
 -> build navigation graph
 -> geometry/topology/semantic validation
 -> review items with confidence/provenance
 -> admin review/edit/reject
 -> approve and publish immutable map version
```

12.6 Student experience

- Search room code, building, facility or place.
- Quick categories may include classrooms, labs, offices, library, washrooms and auditorium.
- Simple 2D map: start, destination pin, route line and concise text directions.
- Show next class and “Navigate to class”.
- If precise location is unavailable, ask user to select start point or entrance.
- Do not claim indoor GPS. QR start points may be a later enhancement.

12.7 Admin map management

- Upload multiple assets and associate with campus/building/floor.
- View processing status, extracted geometry and confidence indicators.
- Review room labels, room matches, entrances, stairs/lifts and graph connectivity.
- Run validation; distinguish blocking errors from warnings.
- Approve/publish only after checks; preserve prior version and rollback.
- Detailed Admin review-screen design was intentionally deferred to a later UI phase.

12.8 Candidate endpoints

```
GET  /api/v1/campus/places/search?q=CR-101
GET  /api/v1/campus/places/{place_id}
POST /api/v1/campus/navigation/route
GET  /api/v1/students/me/next-class
POST /api/v1/admin/campus-map/uploads
POST /api/v1/admin/campus-map/{asset_id}/process
GET  /api/v1/admin/campus-map/review-items
POST /api/v1/admin/campus-map/validate
POST /api/v1/admin/campus-map/approve
POST /api/v1/admin/campus-map/publish
GET  /api/v1/admin/campus-map/versions
POST /api/v1/admin/campus-map/rollback
```

12.9 Safety and limitations

- Unverified geometry cannot be used for public routing.
- Do not route through walls, locked/private areas or inaccessible stairs when accessible route is requested.
- Confidence is a review aid, not proof of correctness.
- If no graph path exists, explain route unavailable; never invent one.
- Check licensing/permission for satellite imagery before publication.

13. UI/UX & Panel Structure

The interface should be clean, modern, responsive and role-aware. Student experience prioritises immediate tasks and navigation; admin experience prioritises queues, conflicts and safe decisions. Avoid decoration that does not help task completion.

Panel	Primary navigation
Admin	Overview; Timetable; Exams; Venue Requests; Disruptions; Campus Map; Analytics; Data Import; Exports; Users & Roles; Audit.
Department Coordinator	Overview; Department Timetable; Faculty/Group coordination; Venue Requests; Events; Notifications.
Club Coordinator	Overview; Create Venue Request; My Requests; Alternatives; Event Details; Notifications.
Faculty	My Schedule; Availability/Leave; Room Request; AI Assistant; Navigate to Class; Notifications.
Student	Today; My Timetable; Exams; Campus Search; Campus Map; Navigate to Class; Notices.

13.1 Screen principles

- Show role-relevant actions; do not expose admin-only navigation to students.
- For every state-changing action show status, next step and responsible role.
- Approval screens show request, conflicts, priority, alternatives and decision reason.
- Timetable supports group/faculty/room filters, week/day navigation and conflict highlighting.
- Map remains simple 2D; avoid decorative 3D complexity.
- Forms validate inline and preserve input after recoverable errors.
- Use accessible labels, keyboard interaction, adequate contrast and clear loading/empty/error states.

13.2 Admin dashboard

- Published timetable status and latest generation result.
- Pending approvals by type and age; active conflicts and unresolved validation errors.
- Upcoming exams and venue events; room utilisation.
- Active disruptions and pending reschedule proposals.
- Campus map version status and unreviewed map items.

13.3 Approval screen minimum information

- Requester identity/role/scope; category; submission time; priority rank.
- Requested dates/times/rooms/capacity/purpose.
- Schedule/resource conflict results; hard blockers vs soft warnings.
- Permitted decision actions; mandatory rejection reason; alternative flow.
- History of changes and previous approvals.

14. Non-Functional Requirements

14.1 Security & privacy

- **NFR-SEC-001:** All protected endpoints enforce authentication and authorisation server-side.
- Never store plaintext passwords/secrets; use trusted auth library/provider.
- Use secure cookies or carefully managed tokens, CSRF protection where applicable and secure expiry.
- Validate file type, size and content; store uploads outside executable public paths.
- Restrict private student/faculty records and source map assets.
- Use parameterised ORM/database operations; validate inputs at boundaries.
- Rate-limit login, AI and expensive job submission.
- Keep secrets out of source code and prompts.
- Audit role changes, approvals, publication, import/export and overrides.

14.2 Performance & reliability

- **NFR-PERF-001:** Normal reads should be responsive under expected hackathon load; set measured targets after hosting is known.
- Long-running solver/map jobs run asynchronously.
- Use pagination and indexes; cache only safe versioned reads.
- Persist job state across browser refresh.
- Back up database and map assets before production.

14.3 Consistency and auditability

- Use transactions for multi-record decisions.
- Published versions are immutable and traceable to dataset/map version.
- Proposals record base version.
- Important actions record actor, timestamp, scope, reason and request ID.
- Use deterministic priority ordering and stable exports.

14.4 Accessibility

- Proposed production target: WCAG 2.2 AA; confirm audit scope.
- Do not encode status by colour alone; support keyboard and visible focus.
- Provide text equivalents for map symbols and route instructions.
- Use clear language and concise errors.

15. Edge Cases, Errors & Recovery

Area	Edge case	Expected behaviour
Import	Missing file, duplicate ID, broken reference.	Reject activation; show precise errors; keep active dataset.
Timetable	No feasible solution.	Do not publish; show INFEASIBLE and diagnostics.
Timetable	Time limit reached.	Show UNKNOWN/FEASIBLE accurately; never mislabel.
Timetable	Dataset changed during solve.	Mark stale; revalidate/re-solve.
Manual edit	Concurrent admin changed same slot.	Return conflict/version mismatch.
Booking	Competing approvals.	Lock/recheck; only one valid allocation commits.
Booking	Self-approval/cancelled/already decided.	Deny or idempotently return current state; audit.
Approval	Reviewer lost scope/role.	Deny using current permissions.
Disruption	No valid reschedule.	Keep current schedule; explain infeasibility.
AI	Missing date/time/attendance.	Ask clarification; do not fabricate.
AI	Invalid room/proposal.	Backend rejects with structured reasons.
Map	OCR misreads room label.	Flag for review; do not create canonical room automatically.
Map	No verified entrance/connection.	Do not claim continuous route.
Map	No graph path.	Return no-route response.
Notification	Provider unavailable.	Persist in-app event; retry external delivery.
Export	Stale/invalid source version.	Block or clearly label version/validation status.

15.1 Error response pattern

```
{
  "error": {
    "code": "ROOM_CAPACITY_INSUFFICIENT",
    "message": "Selected room cannot accommodate attendance.",
    "details": {"room_id":"CR-102","required_capacity":80,"available_capacity":60},
    "request_id": "..."
  }
}
```

15.2 Recovery rules

- Failed proposal must not partially alter published schedule.
- Failed approval must not reserve a room or send success notification.
- Failed map job preserves original asset and permits retry.
- Failed import must not replace active dataset.
- Rollback creates a new publication event; history is never erased.

16. Testing & Acceptance Criteria

Layer	Test scope
Unit	Time rules, role caps, priorities, validators, parser schemas, graph path functions.
Database	Foreign keys, uniqueness, rollback, overlap protections and migrations.
API	Auth, permission scope, payload validation, transitions, pagination and errors.
Solver	Hard constraints, objective weights, status handling and independent validation.
Integration	Booking vs timetable, disruption vs schedule, map-to-room linking.
End-to-end	Role journeys, approval flows, export and navigation.
Regression	Official validator and known sample/change files.

16.1 P0 acceptance criteria

- Fresh environment imports starter dataset without hardcoded sample records.
- Generated timetable contains every required session exactly once and has zero hard violations under official validator.
- Exams obey slots, capacity and group/room conflict rules.
- Plain-English/Hinglish booking becomes editable draft before submission.
- Booking eligibility, priority, tie-break and cancelled status are enforced.
- Self-approval prohibited where required; every decision auditable.
- Disruption proposal minimises moves while preserving hard constraints; application requires approval.
- AI-generated changes are rejected if backend validation fails.
- Role-based views show group/faculty/room schedules, approvals, conflicts and utilisation.
- Required JSON exports generated by application and validated.
- Campus search/route resolves an existing room ID; no false indoor-positioning claims.

16.2 Definition of Done

- Requirement, role permissions and edge cases documented.
- Migration/rollback reviewed where schema changes.
- API contract and errors documented.
- Backend validation and audit implemented.
- Frontend loading, empty, success and error states implemented.
- Tests pass; manual acceptance checklist complete; no unresolved P0 defect.
- Completion report lists files, evidence, known limits and follow-up.

17. Implementation Roadmap & Development Gates

Product-first and contract-first. Database implementation begins after workflows and data ownership are understood; do not create tables from screen mockups alone.

Gate	Work	Exit criteria
G0 PRD	Confirm scope, users, rules, priorities and policy questions.	Team approves baseline; OPEN items have owners.
G1 Architecture	Confirm team skill, framework, deployment and module boundaries.	Architecture decision approved.
G2 Data model	ERD, schema, source mapping, constraints, versioning and migrations.	Schema review; clean DB migration passes.
G3 API/workflows	OpenAPI, state machines, errors and permissions.	Contracts reviewed.
G4 Foundation	Repo, config, auth, RBAC, audit, DB session and health checks.	Role tests pass.
G5 Import	Starter pack import, validation, versioning and source preservation.	Dataset and report verified.
G6 Timetable	CP-SAT, hard/soft rules, validator and exports.	Official validator passes sample/judge changes.
G7 Exams	Exam allocation and validation.	Exam constraints pass.
G8 Bookings	Natural language draft, role cap, priority, approvals, concurrency.	Competing requests/self-approval tests pass.
G9 Disruptions	Proposal, minimal movement, review and apply.	No unapproved mutation; infeasibility handled.
G10 Navigator	Map data, search/route, admin ingestion/review.	Verified graph routes and map checks pass.
G11 AI	Gemini tools, schemas, context and validation loop.	Invalid AI proposals cannot mutate state.
G12 UI	Role panels and complete user journeys.	End-to-end flows pass.
G13 Final	Exports, reset, judge scenarios, deployment and demo.	Fresh environment passes required scenarios.

17.1 Suggested repository structure

```
cit-campus-platform/
  apps/web/                 # React + TypeScript
  backend/app/
    core/                   # config, auth, permissions, database
    modules/
      users/ institutional_data/ timetable/ exams/
      bookings/ approvals/ disruptions/ assistant/
      campus_navigation/ analytics/ exports/ notifications/
    tests/
  backend/migrations/
  worker/                   # solver and map jobs
  docs/
    MASTER_PRD.md
    SYSTEM_ARCHITECTURE.md
    DATABASE_DESIGN.md
    API_SPECIFICATION.md
    WORKFLOWS_AND_APPROVALS.md
    AI_INTEGRATION.md
    UI_UX_SPECIFICATION.md
    TESTING_AND_ACCEPTANCE.md
    IMPLEMENTATION_ROADMAP.md
  scripts/
  exports/
```

17.2 Feature implementation loop

```
1. Read approved requirement and dependencies
2. Inspect repository and existing patterns
3. State plan and affected files
4. Confirm database/API impact
5. Implement backend rules first
6. Add tests and run them
7. Implement frontend against real API
8. Run end-to-end acceptance
9. Report files, evidence and limitations
10. Get team approval before next feature
```

18. Google AI Studio Operating Instructions

Use these as project-level instructions or the first implementation prompt. They are intended to reduce uncontrolled code generation and preserve approved architecture.

```
You are the implementation assistant for the CIT Campus Scheduling &
Venue Management Platform.

SOURCE OF TRUTH
- Follow the approved Master PRD and supporting specifications.
- Do not invent institutional policy or silently change requirements.
- Mark missing decisions OPEN and ask before implementing them.
- Inspect actual starter files and validator rather than guessing.

ARCHITECTURE
- Use approved stack and module boundaries.
- Do not add another backend, database, CMS, microservice, queue or
  framework without explaining the need and obtaining approval.
- Preserve canonical source IDs.
- Use migrations for schema changes.

IMPLEMENTATION
- Work on one feature/task at a time.
- Inspect repository before editing.
- Do not overwrite working code without impact explanation.
- Do not substitute fake data for backend behaviour.
- Keep business rules in backend domain services.
- Follow shared schemas and API contracts.

SAFETY
- Enforce permissions on backend.
- AI cannot approve, publish or bypass rules.
- Validate every solver/AI proposal deterministically.
- Never weaken a hard constraint to pass a test.
- Audit consequential state changes.

FOR EACH TASK REPORT
1. Requirements implemented
2. Assumptions and OPEN questions
3. Files inspected
4. Files created/changed
5. Database/API impact
6. Implementation summary
7. Tests run and exact results
8. Known limitations
9. Confirmation unrelated scope was not changed

Do not start the next feature until acceptance criteria are reviewed.
```

18.1 Task prompt template

```
Implement only: [FEATURE ID + NAME]

Read MASTER_PRD.md and the relevant database, API, workflow,
AI and testing specifications.

Before coding:
- Inspect repository and identify existing patterns.
- List requirements, permissions, dependencies and edge cases.
- Identify schema/API changes.
- Ask about OPEN decisions that block correctness.

Implement backend validation, migration if needed, API/error handling,
frontend flow against the contract and automated tests.

Do not implement unrelated features. Return a completion report
with exact test evidence and known limitations.
```

19. Open Decisions & Confirmation Register

Resolve these during PRD review. They must not be guessed during implementation.

ID	Decision	Current position / next action
OD-01	Team backend skill	OPEN: confirm Python comfort; revisit stack if team cannot debug Python.
OD-02	Backend framework	PROPOSED FastAPI + PostgreSQL; Payload alternative only with explicit Python solver service.
OD-03	Role mapping	PROPOSED six categories; confirm CIT role holders and scopes.
OD-04	Approval chain	OPEN: single/multi-level approvers and delegation by request type.
OD-05	Exam authority	OPEN: assign permission to authorised admin; no invented role.
OD-06	Faculty leave policy	OPEN: approval required or informational?
OD-07	Validator exact rules/weights	OPEN: inspect validator.py and verify recorded weights.
OD-08	Canonical dataset	OPEN: inspect actual starter archive, manifest, JSON and CSV.
OD-09	Map source assets	OPEN: confirm PDFs, scans, images, vector data and licences.
OD-10	Campus georeference	OPEN: obtain verified coordinates and building entrances.
OD-11	Indoor floor-plan calibration	OPEN: confirm scale, orientation and coordinate transform.
OD-12	Start location	OPEN: manual, QR or device location; no indoor GPS promise.
OD-13	Hosting/budget	OPEN: choose provider/region and verify current limits/cost.
OD-14	Authentication	OPEN: local, institutional email, OAuth or SSO.
OD-15	Notifications	OPEN: in-app first; select email/push provider later.
OD-16	Accessibility	PROPOSED WCAG 2.2 AA; define accessible path semantics.
OD-17	Timezone/calendar	OPEN: campus timezone and holiday/academic calendar.
OD-18	Export JSON schema	OPEN: match judge-required format exactly from starter pack.

19.1 Decisions already recorded

- Platform includes scheduling/venue management and Campus Navigator.
- Navigator covers outdoor and indoor navigation; simple 2D experience, not decorative 3D.
- Admin source inputs include PDFs, scans, satellite/map images and existing map data.
- Students search for class/place and receive a route; “Navigate to class” links timetable to room ID.
- Map upload, AI processing and human admin review are intended capabilities.
- Detailed Admin map review-screen UI was deferred to a later design phase.
- Approvals must be considered across state-changing features.
- AI proposals are backend-validated; AI cannot autonomously approve.
- The team wants the complete PRD before feature-by-feature implementation.

20. Glossary

Term	Meaning
Hard constraint	Rule that must never be violated for a valid solution.
Soft constraint	Preference/quality objective optimised after hard feasibility.
CP-SAT	OR-Tools constraint programming/SAT solver.
Dataset version	Immutable snapshot of imported source data/configuration.
Timetable version	Saved assignments based on a particular dataset version.
Disruption	Operational change affecting existing schedule.
Approval record	Formal record of authority, decision, reason and transitions.
Navigation graph	Nodes/edges representing verified walkable paths.
Map version	Immutable published campus geometry, semantics and graph.
Idempotency	Retrying same operation does not create duplicate side effects.
RBAC	Role-based access control combined with resource/scope checks.
Source ID	Identifier from official CIT dataset preserved by the app.

21. External Technical References

These references support the technical recommendations; they do not replace the CIT problem statement, starter data or official validator.

- Payload — What is Payload? Official documentation. Describes Payload as a Next.js full-stack framework with admin panel, migrations, APIs, authentication and access control.
- Payload — PostgreSQL adapter Official documentation.
- FastAPI — SQL databases Official documentation.
- Google OR-Tools — CP-SAT solver Official documentation. Defines OPTIMAL, FEASIBLE, INFEASIBLE, MODEL_INVALID and UNKNOWN statuses.
- Google OR-Tools — Scheduling recipes Official documentation.
- MapLibre GL JS Official documentation.

22. Final Product & Engineering Principle

Build in this order: approve product rules → confirm architecture → design/review database → define API/workflow contracts → backend foundations → import real dataset → timetable → exams/bookings/disruptions → Campus Navigator → constrained AI → role-based UI → judge scenarios and exports.

This is a comprehensive working draft, not final institutional policy. Next: review with the team, resolve architecture/data/approval questions that affect implementation, and freeze the PRD baseline before database implementation.

END OF MASTER PRD — VERSION 1.0 DRAFT